Menyusun recap implementasi login dan sign up dengan JWT. Memeriksa file-file penting:

[6 tools called]

RECAP: Implementasi Login & Sign Up dengan JWT

1. Instalasi dependencies

File: `package.json`

Dependencies yang ditambahkan:

- `bcryptjs` - Password hashing
- `jsonwebtoken` - JWT token generation & verification
- `pg` - PostgreSQL client
- `drizzle-orm` - ORM untuk database
- `zod` - Input validation
- `dotenv` - Environment variables (dev dependency)

Type definitions:

- `@types/bcryptjs`
- `@types/jsonwebtoken`
- `@types/pg`

Scripts yang ditambahkan:

- `db:generate` - Generate migration files
- `db:migrate` - Run migrations
- `db:push` - Push schema ke database
- `db:check` - Check database status

---

2. Database schema

File: `lib/db/schema.ts`

Tabel yang dibuat:

1. `users` table:
    - `id` (UUID, primary key)

    - `email` (VARCHAR, unique, not null)
    - `passwordHash` (VARCHAR, not null)
    - `role` (VARCHAR: 'customer' atau 'penjual')
    - `createdAt`, `updatedAt` (timestamps)

2. `refreshTokens` table:
    - `id` (UUID, primary key)
    - `userId` (UUID, foreign key ke users)
    - `token` (TEXT, unique, not null)
    - `expiresAt` (TIMESTAMP)
    - `createdAt` (TIMESTAMP)

File: `lib/db/index.ts`

- Database connection menggunakan Drizzle ORM
- PostgreSQL connection pool

File: `lib/db/migrations.sql`

- SQL script untuk setup database
- Index untuk performa

---

3. Authentication utilities

File: `lib/auth/jwt.ts`

Fungsi JWT:

- `generateAccessToken()` - Generate access token (15 menit)
- `generateRefreshToken()` - Generate refresh token (7 hari)
- `verifyAccessToken()` - Verify access token
- `verifyRefreshToken()` - Verify refresh token

Secret keys:

- `JWT_SECRET` - Untuk access token
- `JWT_REFRESH_SECRET` - Untuk refresh token

File: `lib/auth/password.ts`

Fungsi password:

- `hashPassword()` - Hash password dengan bcrypt (12 rounds)
- `verifyPassword()` - Verify password dengan hash

File: `lib/auth/middleware.ts`

Helper functions:

- `getTokenFromRequest()` - Ambil token dari request
- `verifyTokenFromRequest()` - Verify token dari request

---

4. API routes

File: `app/api/auth/register/route.ts`

Fitur register:

- Validasi input dengan Zod
- Cek email sudah terdaftar
- Hash password dengan bcrypt
- Generate access token & refresh token
- Simpan refresh token ke database
- Set cookies (httpOnly)
- Return user info

File: `app/api/auth/login/route.ts`

Fitur login:

- Validasi input dengan Zod
- Cari user di database
- Verify password
- Generate access token & refresh token
- Simpan refresh token ke database
- Set cookies (httpOnly)
- Return user info

File: `app/api/auth/logout/route.ts`

Fitur logout:

- Hapus refresh token dari database
- Hapus cookies (accessToken & refreshToken)

File: `app/api/auth/refresh/route.ts`

Fitur refresh token:

- Ambil refresh token dari cookie
- Verify refresh token
- Cek di database (apakah masih valid)
- Generate access token baru
- Update cookie dengan access token baru

File: `app/api/auth/me/route.ts`

Fitur cek auth status:

- Ambil access token dari cookie
- Verify token
- Return user info jika authenticated

---

5. Frontend pages

File: `app/(auth)/login/page.tsx`

Fitur login page:

- Form handling dengan state management
- Validasi input
- Cek auth status saat mount (redirect jika sudah login)
- Error handling & loading states
- Auto-redirect setelah login berhasil
- Success/error message display

File: `app/(auth)/register/page.tsx`

Fitur register page:

- Form handling dengan state management
- Validasi password match
- Validasi password length
- Error handling & loading states
- Success message dengan auto-redirect
- Password confirmation field

---

6. Route protection

File: `middleware.ts`

Fitur middleware:

- Protect routes berdasarkan role
- Seller routes: `/seller/*` (hanya untuk `penjual`)
- Customer routes: `/dashboard/*`, `/cart/*`, `/checkout/*` (untuk `customer` & `penjual`)
- Verify access token dari cookie
- Check authorization berdasarkan role
- Redirect ke login jika tidak authenticated
- Attach user info ke request headers

Protected routes:

- `/seller/:path*` - Hanya penjual
- `/dashboard/:path*` - Customer & penjual
- `/cart/:path*` - Customer & penjual
- `/checkout/:path*` - Customer & penjual

---

7. Header component

File: `components/layout/Header.tsx`

Fitur header:

- Cek auth status saat mount
- Profile dropdown jika sudah login
- User info display (email, role)
- Link ke dashboard (berdasarkan role)
- Logout button
- Auto-refresh auth status saat window focus
- Loading state saat cek auth

---

8. Configuration files

File: `drizzle.config.ts`

- Drizzle ORM configuration
- Schema path, output path, database driver

File: `SETUP_AUTH.md`

- Dokumentasi setup authentication

- Panduan database migration
- Environment variables setup

File: `scripts/check-db.js`

- Script untuk cek database status
- Menampilkan jumlah users
- Menampilkan daftar users

---

9. Environment variables

File: `.env.local` **(perlu dibuat)**

Variables yang diperlukan:

```
DATABASE_URL=postgresql://username:password@localhost:5432/ecoconstruct_c
JWT_SECRET=your-super-secret-jwt-key-min-32-characters
JWT_REFRESH_SECRET=your-super-secret-refresh-key-min-32-characters
NODE_ENV=development
```

Fitur yang diimplementasikan

Authentication

- Login dengan email & password
- Register akun baru
- Logout
- Token-based authentication (JWT)
- Auto-refresh access token

Authorization

- Role-based access control (customer & penjual)
- Protected routes dengan middleware
- Route protection berdasarkan role

Security

- Password hashing dengan bcrypt

- JWT tokens dengan expiration
- HttpOnly cookies
- Token verification
- Refresh token rotation

User experience

- Form validation
- Error handling
- Loading states
- Success messages
- Auto-redirect setelah login/register
- Profile dropdown di header

Flow yang diimplementasikan

Register flow:

```
1. User isi form register
2. Submit → POST /api/auth/register
3. Validasi input
4. Hash password
5. Simpan ke database
6. Generate tokens
7. Set cookies
8. Redirect ke homepage
```

Login flow:

```
1. User isi form login
2. Submit → POST /api/auth/login
3. Validasi input
4. Cek user di database
5. Verify password
6. Generate tokens
7. Set cookies
8. Redirect ke homepage
```

Protected route flow:

```
1. User akses protected route
2. Middleware intercept
3. Ambil access token dari cookie
4. Verify token
5. Check authorization (role)
6. Allow/deny access
```

File structure

```
ecoconstruct-website/
├── app/
│   ├── (auth)/
│   │   ├── login/page.tsx
│   │   └── register/page.tsx
│   └── api/
│       └── auth/
│           ├── login/route.ts
│           ├── register/route.ts
│           ├── logout/route.ts
│           ├── refresh/route.ts
│           └── me/route.ts
├── components/
│   └── layout/
│       └── Header.tsx
├── lib/
│   ├── auth/
│   │   ├── jwt.ts
│   │   ├── password.ts
│   │   └── middleware.ts
│   └── db/
│       ├── schema.ts
│       ├── index.ts
│       └── migrations.sql
├── middleware.ts
├── drizzle.config.ts
├── scripts/
│   └── check-db.js
└── package.json
```

Ringkasan

Anda telah mengimplementasikan:

1. Database schema untuk users & refresh tokens
2. JWT token generation & verification
3. Password hashing & verification
4. Login & register API endpoints
5. Logout & refresh token endpoints
6. Frontend login & register pages
7. Route protection dengan middleware
8. Role-based authorization
9. Profile dropdown di header
10. Error handling & validation

Sistem authentication dan authorization sudah lengkap dan siap digunakan.